

QA Intern Assessment

This is a working assessment, not a quiz. It mirrors a day in our QA team: an imperfect spec, a buggy build, an API, real data, and not enough time. We are scoring how you **think** — clear reasoning beats long answers everywhere below.

Paper code	Time guide	Total marks	Pass benchmark
SKQA-2609-048	~3 hours (self-paced)	100	60 — exceptional answers in ANY two sections also qualify

Ground rules

- Submit ONE document (PDF preferred) as a reply to the email that delivered this. Name it **YourName_YourEmail_SkodeQA.pdf** (example: Ravi_ravi@gmail.com_SkodeQA.pdf).
- Number every answer (A1, A2, B1 ...). Unanswered is fine; padded is not.
- **Do this yourself.** Search and documentation are fine; AI assistants are NOT. This assessment exists to see how YOU think — an AI-written submission tells us nothing and is disqualified.
- **We validate.** Your paper is a unique variant (SKQA-2609-048): its numbers differ from every other candidate's, every submission is cross-checked against other candidates and against AI-generated baseline answers we produce ourselves, and your answer sheet must carry your paper code.
- **Defense round.** Shortlisted candidates re-solve one item live on a short call and answer follow-ups on their own submission. A strong sheet you cannot defend counts against you, not for you.
- Where a spec is silent, state your assumption and continue — that is part of the test.

Your test environment — the real Skode CRM

- Create a **free account** at <https://skode.ai/signup> using the SAME email you applied with, and sign in to the CRM at <https://crm.skode.ai>.
- Name your organisation/company exactly your paper code: **SKQA-2609-048**. We verify hands-on work server-side — a submission with no matching activity in its own account is treated as not attempted.
- The written specs below are the PM's INTENT. Where a question says "verify live", run it against the real CRM (Leads module) inside YOUR account and report what you actually observe — matches, mismatches and surprises all earn marks.
- **Fair use:** this is a live product. Stay inside your own account, form-level testing only — no load testing, scanning, or attacks beyond what a question explicitly asks. Breaking this rule ends the application.

Section A — Test design from an imperfect spec (25 marks)

Feature spec — "Add Lead" form:

Field	Rules (as written by the product manager)
Full name	Required. 2–80 characters.
Email	Required. Must be a valid email. Unique per organisation.
Phone	Optional. 10 digits. Indian numbers only.
Lead value (Rs.)	Optional. Positive number up to 2,00,00,000.

Source	One of: Website, Referral, Ads, Other.
Save button	Creates the lead and shows it in the list within 4 seconds.

A1. (10) Write your **ten highest-value test cases** for this form — not the first ten you think of. Each: ID, title, steps (brief), test data, expected result. Cover positive, negative and boundary. **Verify live:** execute at least six of them against the real Add Lead form in your account and record the OBSERVED result beside the expected one.

A2. (8) The spec above is ambiguous or incomplete in at least **five** places. List every gap you can find, and for each write the ONE question you would ask the product manager.

A3. (7) The PM answers: "Email uniqueness is case-insensitive, and leads can also arrive from a public web form with the same rules." What NEW risks and test cases does the public form introduce? Name at least four, including one security-related.

Section B — Bug hunt (20 marks)

A tester recorded this session against the build. The spec is Section A's, including the PM's answer in A3.

#	What the tester did	What happened
1	Saved a lead with name "Jo"	Error: "Name must be at least 2 characters"
2	Saved with email RAVI@ACME.COM; org already has ravi@acme.com	Lead created
3	Saved with phone "+91 98765 43210"	Error: "Phone must be 10 digits"
4	Saved with lead value 0	Lead created with value Rs.0
5	Saved with value 2,00,00,000	Error: "Value too large"
6	Double-clicked Save once (fast)	Two identical leads in the list
7	Saved a valid lead on slow 3G	Lead appears after 6 seconds; no feedback meanwhile
8	Typed <code><script>alert(1)</script></code> as the name via the public form	Alert popup fired on the leads list page

B1. (12) For each row: is it a defect, expected behaviour, or "cannot say from the spec"? One line of justification each. For every defect, give severity (S1–S4) and priority (P1–P4). **Verify live:** where the real CRM lets you, reproduce the row in your own account and note what YOU observed — the live product may behave differently from this recorded session, and saying so earns marks.

B2. (5) Rank the defects you found in the order you would want them FIXED for a release going out tomorrow, and defend the order in ≤ 4 sentences.

B3. (3) Row 8 is a vulnerability class. Name it, explain the harm in one sentence, and state one way the developer should fix it.

Section C — The bug report (10 marks)

C1. Pick the WORST defect from Section B and write one complete, developer-ready bug report: title, environment, preconditions, steps, expected vs actual, severity/priority with reasoning, and what you would attach as evidence. It will be scored on whether a developer could reproduce it without asking you anything.

Section D — API testing (20 marks)

The same feature has an API — **specification only**. This endpoint is NOT deployed (the .example domain does not resolve), so do not try to call it: this section is scored on the tests you DESIGN and the commands you WOULD run.

Spec	Detail
Endpoint	POST https://api.skodecrm.example/v1/leads
Auth	Header Authorization: Bearer <token>
Body (JSON)	{"name": "...", "email": "...", "phone": "...", "value": 50000, "source": "Website"}
Success	201 with the created lead including its id
Errors	400 invalid body · 401 bad token · 409 duplicate email · 429 rate limited

D1. (8) Design a test matrix for this endpoint: at least **twelve** cases as rows (input → expected status + body). Boundaries, auth, duplicates, malformed JSON and wrong content-type count.

D2. (6) Write the exact **curl** commands for three of your cases: the happy path, the duplicate-email case, and one malformed-body case.

D3. (6) You call the API twice with identical data because your network retried. Both return 201. Is that a bug? Explain the concept involved, who it hurts in a CRM, and one design that prevents it.

Section E — Data checks with SQL (15 marks)

Table **leads**(id, org_id, name, email, phone, value, source, created_at):

E1. (5) Write the query that finds duplicate emails within the same organisation (email, org_id, count) — case-insensitive.

E2. (5) Write the query for: leads created in the last 7 days with value ≥ Rs.50,000, newest first, showing name, email, value.

E3. (5) After a migration, total lead count matches but SUM(value) dropped by exactly the total of one day's leads. Give your two most likely hypotheses and the ONE query you would run first to decide between them.

Section F — Judgment under pressure (10 marks)

F1. (5) Release is in 4 hours. You have 60 untested cases: 20 payment, 20 lead-form, 20 cosmetic/UI. You can run about 25. Which do you run, in what order, and what do you tell the release manager about what was NOT covered? ≤8 sentences.

F2. (5) A developer closes your defect from B row 6 as "works as designed — users should not double-click". Write your ≤4-sentence reply. You may agree or disagree; you are scored on reasoning and tone.

How we score	
Reasoning & clarity	Half the marks in every section. Ten sharp test cases beat thirty shallow ones.

Honesty	"I do not know, but here is how I would find out" earns marks. Bluffing loses them.
Evidence of craft	Assumptions stated, edge cases named, defects argued — like a tester we would trust.

Good hunting. — Skode Careers · careers@skode.ai · skode.ai